

NHẬN DẠNG CHỮ SỐ VIẾT TAY BẰNG THUẬT TOÁN KNN

Sinh viên thực hiện: Nguyễn Thành Trung
Mã số SV: 170122293
Mã lớp: DX22TT8016
Đại học Trà Vinh

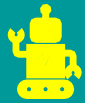


TỔNG QUAN DỰ ÁN



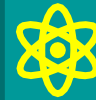
Mục tiêu

Xây dựng hệ thống phân loại chữ số viết tay từ 0 đến 9 với độ chính xác cao dựa trên tập dữ liệu chuẩn MNIST.



Công nghệ

Sử dụng ngôn ngữ Python kết hợp các thư viện Machine Learning mạnh mẽ như Scikit-learn, Numpy, Matplotlib, Pandas, Seaborn, Openpyxl.



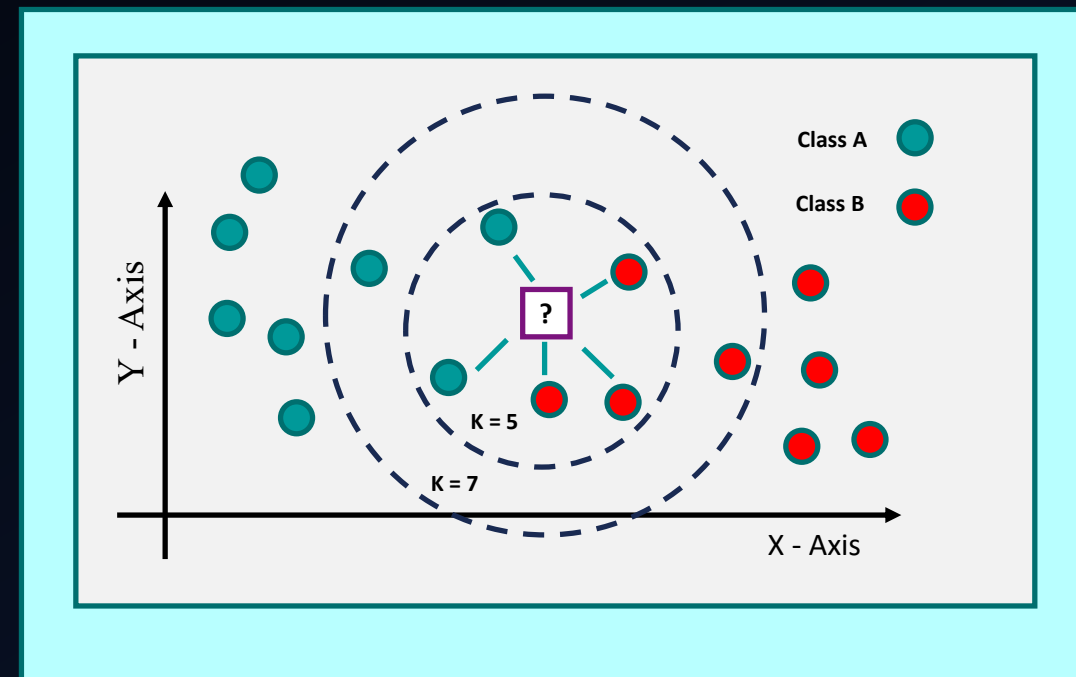
Giải thuật

Áp dụng thuật toán KNN – một giải thuật học máy có giám sát đơn giản nhưng hiệu quả.

THUẬT TOÁN K – NEAREST NEIGHBORS

Nguyên lý: Phân loại một điểm dữ liệu mới dựa trên đa số phiếu bầu của K điểm gần nó nhất trong không gian đặc trưng.

- ✓ Tính khoảng cách (Euclidean).
- ✓ Sắp xếp và chọn ra K láng giềng gần nhất.
- ✓ Gán nhãn phổ biến nhất cho điểm cần phân loại.
- ✓ Không cần giai đoạn huấn luyện (Lazy Learning).



DỮ LIỆU MNIST

MNIST là tập dữ liệu chữ số viết tay chuẩn trong thị giác máy tính.



70k Ảnh grayscale chữ số 0-9.



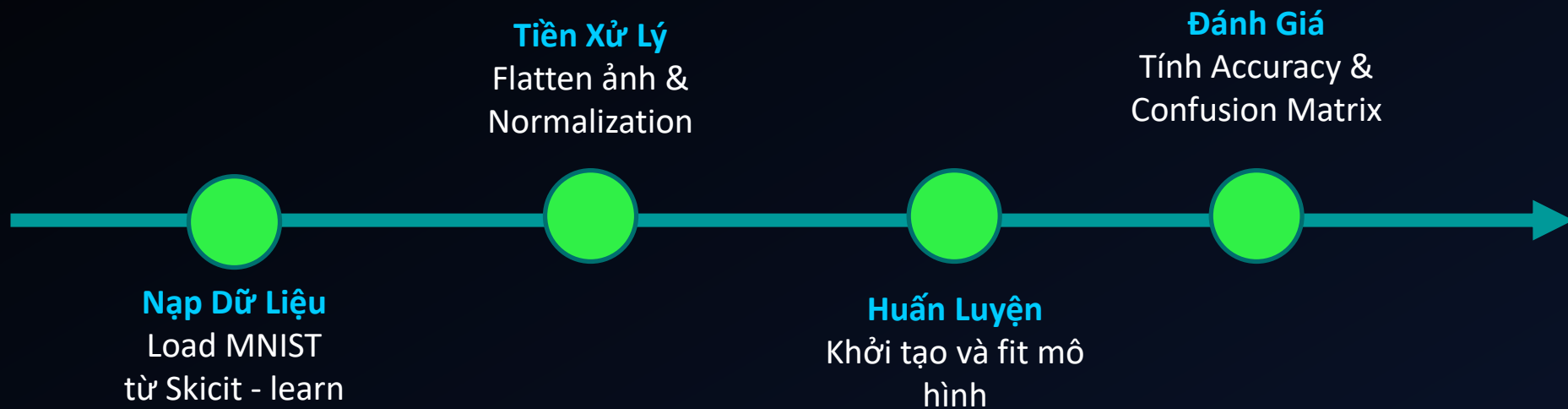
Kích thước mỗi ảnh: 28 x 28 pixels.



54k ảnh huấn luyện, 14k ảnh kiểm tra.



QUY TRÌNH THỰC HIỆN



TIỀN XỬ LÝ DỮ LIỆU

Chuyển đổi kích thước



Khi dùng **fetch_openml** ('mnist_784'): Thư viện này mặc định trả về dữ liệu X ở dạng một **DataFrame** hoặc **Numpy** mảng đã được làm phẳng sẵn thành vector 784 chiều ($28 \times 28 = 784$).

```
from sklearn.datasets import fetch_openml
```

Chuẩn hóa (Scaling)



Chuyển giá trị điểm ảnh từ khoảng [0, 255] về khoảng [0, 1] giúp thuật toán tính khoảng cách chính xác và nhanh hơn.

$$X' = \frac{X}{255}$$

XÂY DỰNG MÔ HÌNH



Khởi tạo

Sử dụng `KNeighborsClassifier` với tham số `n_neighbors` và `metric='minkowski'` (Euclidean).



Huấn luyện

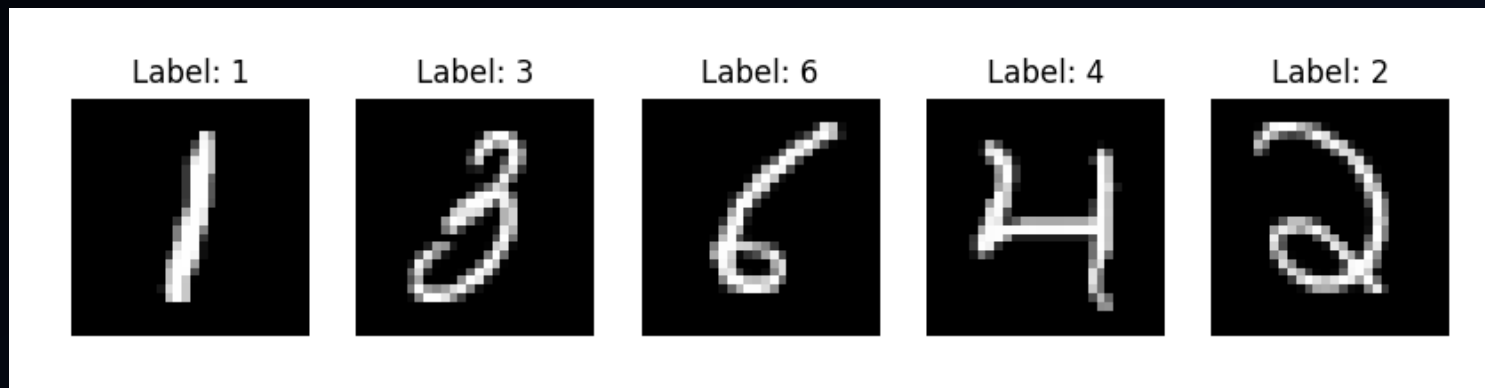
Mô hình lưu trữ toàn bộ tập dữ liệu huấn luyện để làm cơ sở so sánh khoảng cách khi dự đoán.



Tối ưu

Thử nghiệm các giá trị `K` khác nhau để tìm ra cấu hình cho độ chính xác cao nhất trên tập `Validation`.

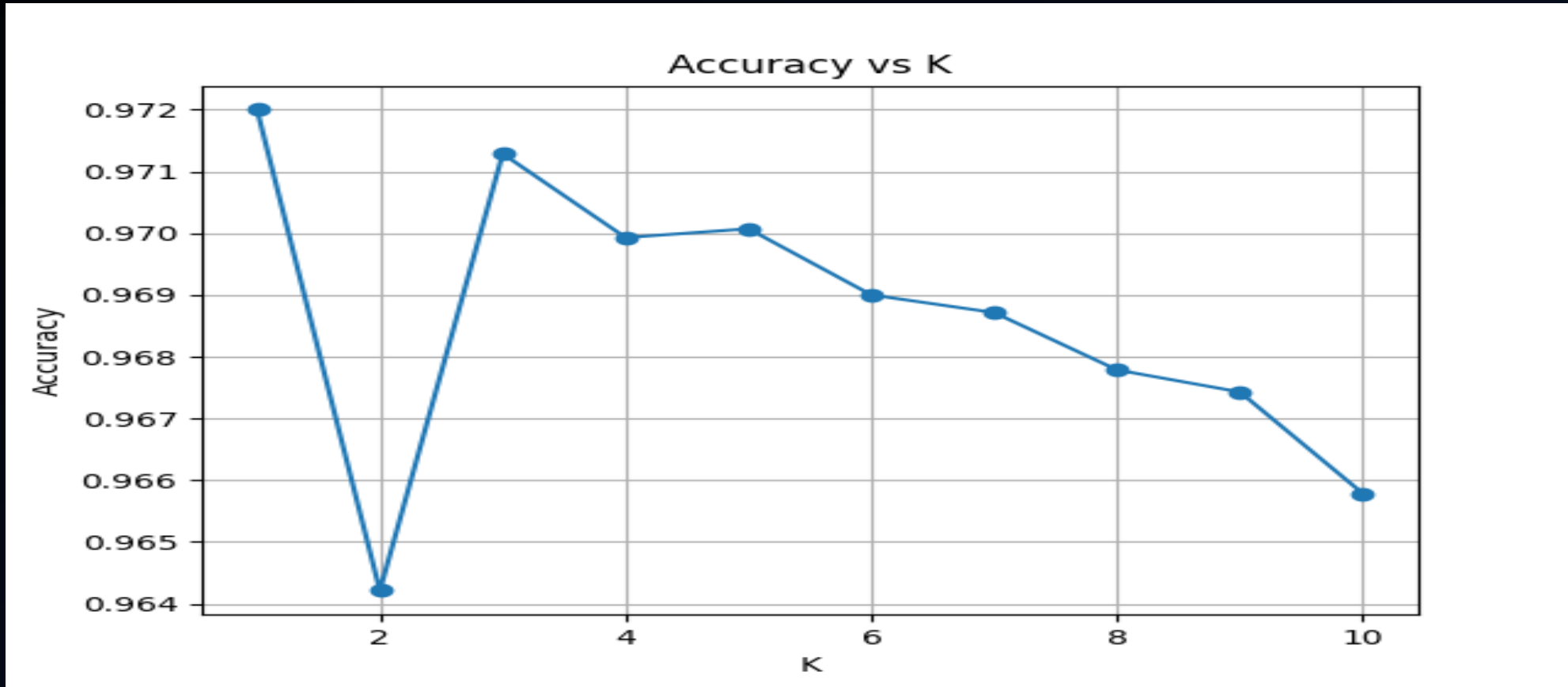
XUẤT RA 5 MẪU NGẪU NHIÊN KHÔNG TRÙNG LẶP



```
random_indices = np.random.choice(len(X), 5, replace=False)
```

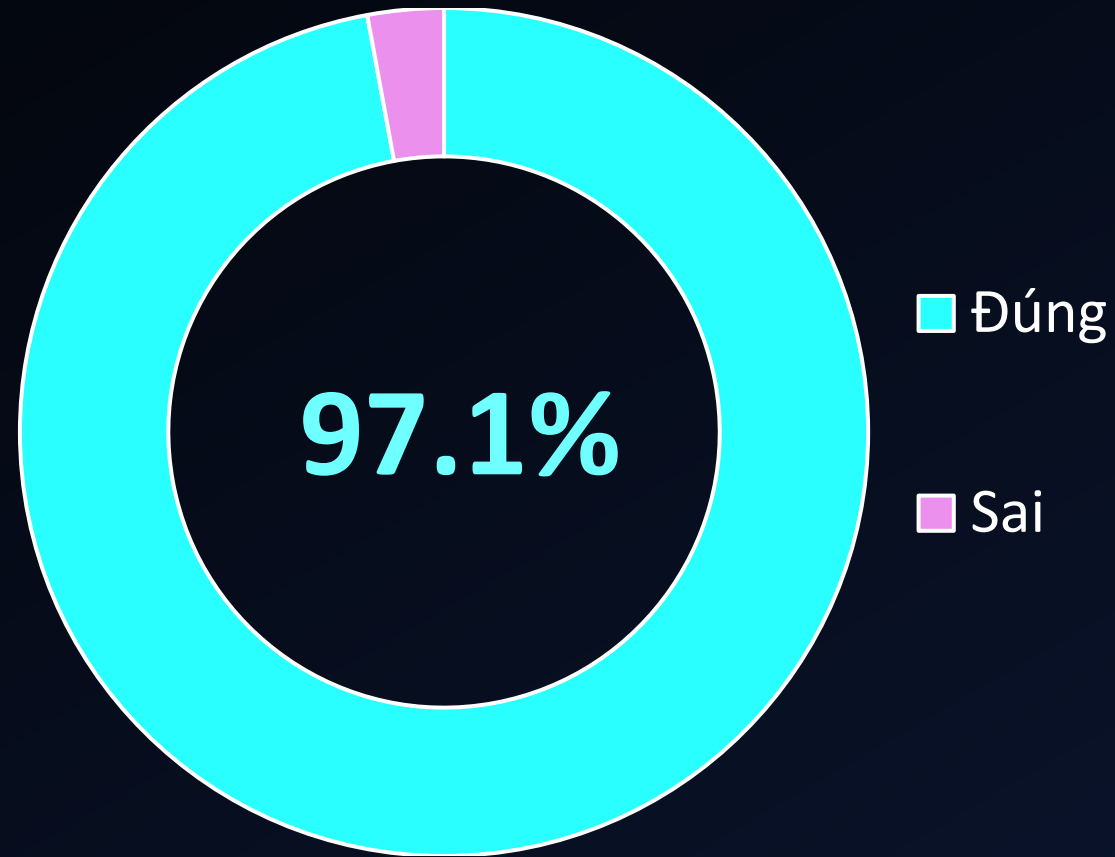
→ Lấy ra mẫu khách quan: Mỗi lần chạy code sẽ lấy ra 5 mẫu khác nhau.

TỐI ƯU HÓA THAM SỐ K



=> Kết quả cho thấy tuy $K = 1$ cho độ chính xác cao nhất, nhưng mô hình có thể nhạy với nhiễu (noise) và dễ bị overfitting. Trong thực tế, các giá trị K lớn hơn (như 3 hoặc 5) thường được cân nhắc để đảm bảo tính ổn định.

ĐỘ CHÍNH XÁC CỦA KNN ($K = 3$)

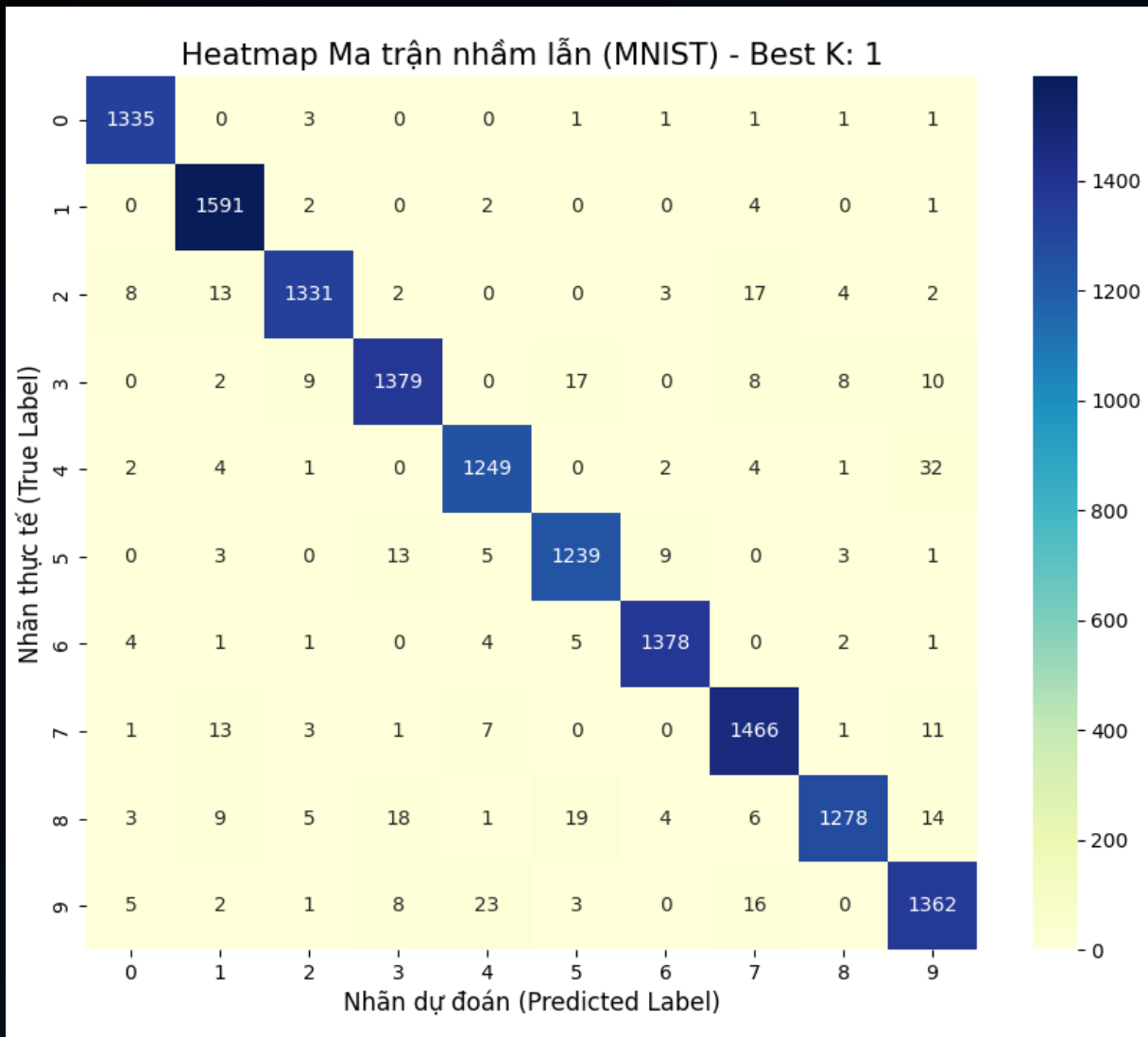


KẾT QUẢ ĐẠT ĐƯỢC

Chữ Số	Precision	Recall	F1 - Score	Accuracy
Số 0, 1, 2...	97 – 98%	96 – 99 %	97%	97.2%
Trung bình	97%	97%	97%	

==> Model đạt accuracy ~97.2%, cùng với precision, recall và F1-score đều xấp xỉ 97%. Điều này cho thấy mô hình hoạt động ổn định và không có dấu hiệu lệch lớp rõ rệt. Với thuật toán KNN, mức Accuracy trên ~97% được xem là kết quả tốt trên tập MNIST, đặc biệt khi chưa áp dụng các kỹ thuật tối ưu nâng cao.

PHÂN TÍCH MA TRẬN NHẦM LẦN (CONFUSION MATRIX)



Một số chữ số dễ bị nhầm lẫn do hình dạng tương tự.

Ví dụ:

- 4 ↔ 9
- 7 ↔ 1
- 8 ↔ 3

=> Điều này xảy ra vì dữ liệu viết tay không đồng nhất.

TỔNG KẾT & ĐÁNH GIÁ



Ưu điểm: KNN cực kỳ hiệu quả với bộ dữ liệu MNIST, cho độ chính xác cao mà không cần kiến trúc phức tạp.



Nhược điểm: Tốc độ dự đoán chậm khi tập dữ liệu lớn do phải tính toán khoảng cách với mọi điểm huấn luyện.



Hướng phát triển: Áp dụng PCA (Giảm chiều dữ liệu) để tăng tốc độ tính toán hoặc sử dụng CNN (Mạng nơ-ron tích chập) để đạt độ chính xác >99%.

XIN CẢM ƠN THẦY CÔ & CÁC BẠN ĐÃ XEM!

Sinh viên thực hiện: Nguyễn Thành Trung
Mã số SV: 170122293
Mã lớp: DX22TT8016
Đại học Trà Vinh

Thông tin liên hệ
Mail: 170122293@rdi.edu.vn
Phone: 0816.221.299